

- A: https://oj.uz/problem/view/IOI17_coins?locale=en
 B: <https://codeforces.com/problemset/problem/1697/D?locale=en>
 C: <https://qoj.ac/problem/4830>
 D: <https://qoj.ac/problem/7156>
 E: https://oj.uz/problem/view/COI18_zagonetka?locale=en

Semana 9: soluciones (por Innokentiy Kaurov)

A. Necesitamos codificar la posición de la moneda mala en la configuración del tablero. Entonces, necesitamos una función hash que tome un tablero y devuelva un solo número. ¡Piensa en algo binario! Definamos $f(b)$ como el **XOR** ($\oplus$) de todas las posiciones i con $b_i = 1$. Voltear la moneda i corresponde a reemplazar $f(b)$ por $f(b') = f(b) \oplus i$. En la función `coin_flips`, transformemos b en b' tal que $f(b') = c$, donde c es la posición de la moneda mala. Para lograrlo, hay que elegir i tal que $f(b) \oplus i = c$. Entonces, simplemente volteamos la moneda $i = f(b) \oplus c$. En la función `find_coin`, devolvemos $f(b)$.

B. Sea $q_1(i)$ la primera consulta y $q_2(l, r)$ la segunda consulta. Reconstruyamos la cadena de izquierda a derecha. Supongamos que sabemos todos los caracteres antes de i y queremos saber s_i . Encontremos la ocurrencia más reciente de s_i . Sea $d(l, r)$ el número de caracteres distintos en $s[l..r]$, donde $l \leq r < i$. Según nuestra suposición, todos estos $d(l, r)$ son conocidos.

Fijemos $j < i$. Si $q_2(j, i) = d(j, i - 1)$, entonces la ocurrencia más reciente de s_i está en o después de la posición j . Si $q_2(j, i) = d(j, i - 1) + 1$, entonces la ocurrencia más reciente de s_i está antes de j . Así que podemos iterar sobre j desde $i - 1$ hasta 1 y encontrar la primera posición para la cual $q_2(j, i) = d(j, i - 1)$. En este caso, sabemos que $s_i = s_j$. Si no existe tal j , entonces s_i aparece por primera vez, por lo que podemos usar $q_1(i)$ para obtener su valor.

Este algoritmo utiliza $O(n^2)$ consultas q_2 . Para mejorarlo, podemos realizar una búsqueda binaria sobre j . Así, el número de consultas se reduce a $O(n \log n)$, pero esto sigue siendo demasiadas. La última observación: el índice final j siempre será la ocurrencia más reciente de algún carácter, así que podemos hacer la búsqueda binaria solo sobre las últimas ocurrencias de los 26 caracteres posibles. Este algoritmo resuelve el problema en $n \log 26$ consultas.

C. Anya necesita transferir información sobre el conjunto de interruptores activos a Andrei. Por supuesto, no se puede codificar todos los 2^{10^6} conjuntos posibles utilizando solo 1000 caracteres. Entonces, usemos la técnica estándar llamada “hashing de conjuntos”: Anya va a calcular un solo número que representa el conjunto (el *hash*) y enviarlo a Andrei. El hash va a ser una cadena de K bits, donde K es fijo. Para cada interruptor i , asignamos un hash aleatorio $\text{hash}(i)$ de K bits. Tanto Anya como Andrei deben conocer esta asignación, por lo que necesitan usar el mismo generador aleatorio para crear los hashes. Para un conjunto de interruptores S , definimos $\text{hash}(S) = \text{XOR de } \text{hash}(i) \text{ para todos } i \in S$ (nota la similitud con el problema A). El beneficio de tal hash es que el volteo del estado del interruptor i cambia el hash de manera predecible: $\text{hash}(S') = \text{hash}(S) \oplus \text{hash}(i)$.

Entonces, Anya envía el hash del conjunto final de interruptores activos a Andrei. En cada consulta, Andrei modifica este hash realizando un XOR con $\text{hash}(i)$. Después de cada modificación, ¿cómo se puede entender si el conjunto actual está vacío? Comprobamos si $\text{hash}(S) = 0$. ¿Cómo se puede saber si el conjunto actual tiene solo un interruptor? Comprobamos si $\text{hash}(S) = \text{hash}(j)$ para algún interruptor j . Esto se puede hacer de manera eficiente usando un mapa hash (*unordered_map*). Si ninguno de los casos es detectado, el conjunto debe tener al menos 2 interruptores.

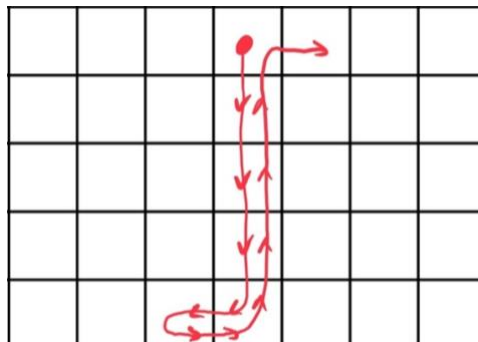
¿Cuál debería ser el tamaño del hash K ? Nuestro algoritmo puede sufrir de colisiones de hash, que ocurren principalmente cuando $|S| > 1$ pero $\text{hash}(S) = \text{hash}(j)$ para algún j . En este caso, el algoritmo erróneamente dice que $|S| = 1$. Entonces, la probabilidad de fallo en cada consulta es aproximadamente $\frac{10^6}{2^K}$ (la probabilidad de que un hash aleatorio coincida con algún $\text{hash}(j)$). Así, el número esperado de fallos durante el algoritmo es $\frac{10^5 \cdot 10^6}{2^K}$. Al fijar $K = 60$, tenemos una buena probabilidad de éxito y los hashes se pueden almacenar cómodamente en el *long long*.

D. Comencemos resolviendo este problema con 50 consultas. Lo haremos comprobando cada columna desde 0 hasta $W - 1$. Para comprobar la columna c , haremos c pasos hacia la derecha y H pasos hacia abajo. Si la caja no está en esta columna, el robot terminará en $(H - 1, c)$. De lo contrario, si la caja está en alguna celda (r, c) con $r \neq 0$, el robot terminará en $(r - 1, c)$. Finalmente, si la caja está en la celda $(0, c)$, el robot terminará en $(H - 1, c - 1)$. Por lo tanto, con una consulta podemos saber si la caja está o no en la columna c y, si está, podemos expresar su posición exacta.

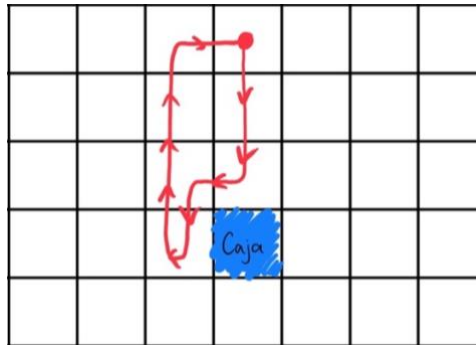
Ahora, mejoremos este algoritmo. Vamos a comprobar todas las columnas en solo una consulta. Para hacer esto, nuevamente vamos comprobando columna por columna y nos gustaría que el robot se quede atascado de alguna manera cuando golpee la caja. Por ahora, supongamos que la caja no está en la primera fila ni en la primera columna. Comenzaremos a comprobar las columnas desde $c = 1$ hasta $W - 1$, por lo que daremos un paso hacia la derecha al principio. Luego:

- Supongamos que queremos comprobar la columna $c \geq 1$ y ahora estamos en $(0, c)$.
- Damos H pasos hacia abajo, luego 1 paso hacia la izquierda, 1 paso hacia abajo, 1 paso hacia la derecha, H pasos hacia arriba y 1 paso hacia la derecha.

Si la columna c está vacía, entonces, después de estos pasos, el robot simplemente terminará en $(0, c + 1)$ y ejecutaremos los mismos pasos para la siguiente columna. La ilustración de abajo muestra este caso.



Sin embargo, si la caja está en una celda (r, c) , entonces el robot terminará en $(0, c)$. La ilustración de abajo muestra este caso.



Todos los pasos sucesivos para $c + 1, c + 2, \dots$ seguirán trayendo al robot a $(0, c)$, y esta será su posición final. Luego, moveremos el robot H pasos más hacia abajo: así, terminaremos en $(r - 1, c)$, desde donde sabremos la posición de la caja. Entonces, este algoritmo nos permite determinar la posición exacta de la caja si no está en la primera fila ni en la primera columna usando solo una consulta.

¿Qué ocurre en el algoritmo cuando la caja está en la primera fila, es decir, en $(0, c)$ para algún $c \geq 1$? Si $c > 1$, el robot también se atascará y terminará en $(H - 1, c - 1)$. Si $c = 1$, terminará en $(H - 1, W - 1)$. El robot no puede terminar en la fila $H - 1$ si la caja no está en la primera fila. Por eso, si vemos que el robot termina en la fila $H - 1$, podemos únicamente determinar la posición de la caja en la primera fila.

Entonces, con una sola consulta, entendemos la posición de la caja si no está en la primera columna. En la solución final para $Q = 2$, vamos a usar una consulta para buscar la caja en la primera columna: si no la encontramos, realizamos la consulta descrita anteriormente para encontrar su posición. Ten cuidado con casos extremos como $H = 1$ o $W = 1$, pero en general, esta solución es muy fácil de implementar.

Este problema tiene muchas soluciones diferentes. Se pueden encontrar soluciones alternativas [aquí](#).

E. Observamos que el número máximo de consultas es aproximadamente $\binom{n}{2}$, que nos sugiere expresar la condición entre cada par de índices i y j . La condición tiene tres estados posibles: $p_i < p_j \Leftrightarrow (i, j)$, $p_j < p_i \Leftrightarrow (j, i)$ o sin restricción. Necesitamos entender cuál se aplica para cada par. Una vez que expresemos las condiciones, encontrar la permutación lexicográficamente mínima o máxima que las satisfaga es un problema independiente.

Parte 1: determinar las condiciones

Es imposible expresar las condiciones ocultas exactas: por ejemplo, los conjuntos de condiciones $\{(1, 2), (2, 3)\}$ y $\{(1, 2), (2, 3), (1, 3)\}$ son indistinguible, porque $(1, 2)$ y $(2, 3)$ implican $(1, 3)$. En vez, vamos a expresar la *clausura transitiva* de las condiciones ocultas: es decir, si hay condiciones (i, j) y (j, k) , digamos que la condición (i, k) existe automáticamente.

Hay que usar el hecho que la permutación p proporcionada es válida. Para cada par i y j con $p_i < p_j$, ya sabemos que la condición (j, i) es imposible: solo hay que comprobar si existe o no la condición (i, j) . ¿Para qué pares i y j es fácil hacerlo?

Sea pos_k la posición de k en p : es decir, $p_{pos_k} = k$. Es fácil comprobar la condición entre pos_k y pos_{k+1} para todos k de 1 a $n - 1$: vamos a intercambiar los dos elementos y preguntar si la permutación nueva es válida o no. Si es válida, entonces no hay ninguna restricción entre pos_k y pos_{k+1} . Si no es válida, entonces el único par problemático es pos_k y pos_{k+1} , porque el orden relativo de los otros elementos permanece sin cambios: así, sabemos que la condición (pos_k, pos_{k+1}) existe.

¿Puedes explicar porque la misma estrategia no se puede usar para un par i y j arbitrario? Si intercambiamos p_i y p_j , entonces también afectamos todos los pares entre i o j y las posiciones k con $p_i < p_k < p_j$. Por eso, en un caso invalido, no podemos determinar el par exacto que causa la contradicción.

Vamos a modificar el algoritmo y reformular el problema usando la permutación pos (la permutación inversa de p). La condición $p_i < p_j$ es equivalente a “el elemento i aparece antes del elemento j en pos ”. Entonces, el método ya mencionado comprueba la permutación formada mediante un intercambio de dos elementos adyacentes en pos , que nos permite encontrar las condiciones entre todos los pares pos_k y pos_{k+1} . Ahora, para un k fijo, encontremos las condiciones entre pos_k y $pos_{k+2}, pos_{k+3}, \dots, pos_n$. Supone que nos interesa la condición entre pos_k y pos_l , donde $k < l$, y ya sabemos todas las condiciones entre pos_k y pos_r para $k < r < l$. Dividimos todos los elementos pos_r para $k < r < l$ en dos conjuntos A y B : $pos_r \in A$ si la condición (pos_k, pos_r) **no existe** y $pos_r \in B$ si la condición (pos_k, pos_r) **existe**.

Observación 1: si $i \in A$ y $j \in B$, entonces no existe la condición (j, i) .

Prueba: Supone que (j, i) existe. Ya que $j \in B$, (pos_k, j) existe también. Ya que las condiciones son transitivas, (pos_k, i) existe, pero esto contradice $i \in A$.

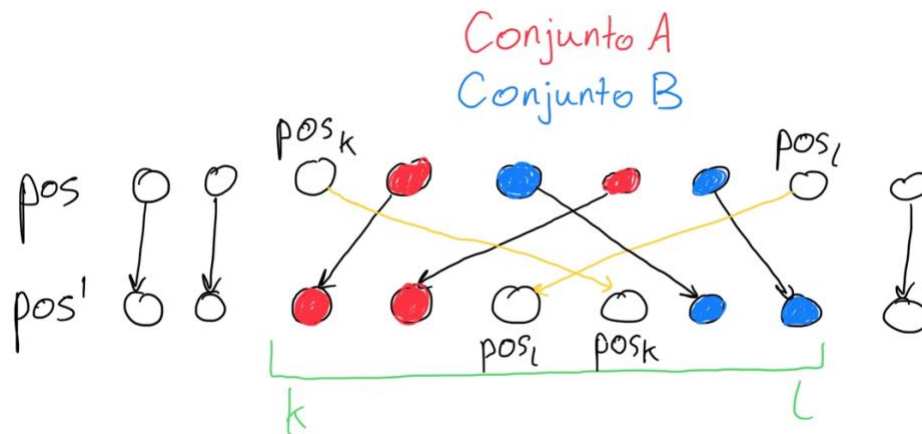
Observación 2: si $j \in B$ y (j, pos_l) existe, entonces (pos_k, pos_l) existe también.

Prueba: según la propiedad transitiva.

Estas observaciones nos permiten comprobar la existencia de (pos_k, pos_l) :

- En las siguientes instrucciones trabajamos con la permutación pos , **no** p .
- Solo vamos a modificar el segmento $[k, l]$ en pos .
- Movemos todos los elementos de A al principio del segmento y todos los elementos de B al final del segmento en el mismo orden.
- Ponemos pos_l y pos_k en este orden entre A y B .
- Obtenemos una permutación inversa pos' y su permutación correspondiente p' . Preguntamos si p' es válida o no.
- Si p' es válida, (pos_k, pos_l) no existe.
- Si no, (pos_k, pos_l) existe.

La visualización de la construcción está en la siguiente página.



¿Por qué funciona este método?

- Si p' es válida, (pos_k, pos_l) no existe porque su orden fue invertido.
- Si p' no es válida, ¿qué pares podrían causar una contradicción en pos' ?
 - Según la observación 1, no puede haber ninguna contradicción entre A y B .
 - No puede haber ninguna contradicción dentro de A o B porque el orden de sus elementos no cambia.
 - Según las definiciones, no puede haber ninguna contradicción entre A y pos_k o pos_k y B .
 - No puede haber ninguna contradicción entre pos_l y A porque su orden no cambia.
 - **Puede haber** una contradicción entre pos_l y pos_k solo si (pos_k, pos_l) existe.
 - **Puede haber** una contradicción entre pos_l y B , pero, según la observación 2, esto implica que (pos_k, pos_l) existe.

En todos los casos, la existencia de una contradicción implica la condición (pos_k, pos_l) . Entonces, podemos comprobar la condición entre pos_k y pos_l para todos $k < l$, lo que resuelve la primera parte del problema.

Parte 2: encontrar la permutación mínima/máxima

En la parte anterior, hemos obtenido el conjunto de condiciones que forman la clausura transitiva de las condiciones ocultas.

Interpretemos las condiciones como un grafo dirigido (un DAG), donde la condición (i, j) corresponde a la arista $j \rightarrow i$.

Para construir la permutación lexicográficamente mínima, usemos el DFS:

- Sea $current = 1$ un variable global.
- Para cada posición i del 1 al n en este orden, ejecutamos un DFS desde i si i aún no ha sido visitada.
- Dentro del DFS para una posición j :
 - Marcamos j como visitada.
 - Iteramos sobre todas las aristas $j \rightarrow k$ en el **orden creciente** de k y, si k no ha sido visitada, ejecutamos el DFS desde k .
 - Después de procesar todas las aristas desde j , asignamos $p_j := current$ y incrementamos $current$.

Este algoritmo greedy se puede probar con la inducción. De hecho, es correcto solo si las aristas en el grafo son transitivas, lo que está garantizado por la primera parte.

Encontrar la permutación máxima es un problema muy similar: hay que trasponer el grafo (invertir las direcciones de las aristas) y ejecutar el mismo DFS, excepto *current* se decrementa desde n .